# College Mentorship Management System (CMMS): A Digital Platform for Faculty–Student Mentorship

Aaditya Bansal
1024030768
Jessica
1024030761
Harshveer Singh
1024030776

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology

Submitted to: Jeelani Asif
Course: UCS503P
August 18, 2026

## 1 Introduction and Problem Statement

Faculty mentorship is one of the biggest factors in whether a student does well in college. A good mentor helps a student pick the right courses, points them toward research or projects that match their interests, and notices early if something is going wrong. Studies on student retention consistently show that students who meet regularly with a faculty mentor are more likely to stay on track and finish strong. Departments also rely on mentorship to identify students who need extra support before small problems turn into failed semesters.

Despite how important this is, most universities still run mentorship in a completely informal way. There is no dedicated system for it — students email professors to ask for a meeting, office hours are unscheduled and often missed, and nothing about the conversation gets recorded anywhere. This causes three specific problems. First, coordination is manual: meetings get missed or forgotten because everything runs through ad hoc emails instead of a shared calendar. Second, there is no tracking: department heads have no way to see how often mentors and students are actually meeting, or whether a student's progress is improving. Third, faculty workload can end up unbalanced, since there is no shared system to see how many students each mentor is already handling across a department with hundreds of students.

CMMS is proposed to close this gap. It is a web-based platform that digitizes the core mentorship process — matching students to mentors, scheduling meetings, logging what happened, and giving department admins visibility into activity — so that mentorship stops depending on informal effort and starts running on a shared, visible system.

## 2 Objectives

The primary goal of this project is to build a web platform that digitizes, schedules, and tracks faculty–student mentorship across university departments, replacing today's informal, email-based coordination with one structured and visible system.

Specific objectives:

- Automatically match students to faculty mentors based on department and stated academic focus area.
- Provide a built-in scheduling system for office hours, with conflict-free booking and every meeting automatically logged.
- Give students, faculty, and department admins separate dashboards, each showing only what that role actually needs to see.
- Let faculty record a short note after each meeting, building a visible history of contact per student over the semester.
- Give department admins a summary view of mentorship activity, so workload and engagement are visible without asking faculty manually.

## 3 Methodology

CMMS is a software engineering project, so the methodology here is the system design: the technology used, how the application is broken into modules, how those modules work together, and how the finished system will be tested.

### 3.1 System Architecture and Tech Stack

The system is built on the MERN stack, in a standard three-tier structure:

- **Frontend (React):** A responsive web interface with three separate views — one each for students, faculty, and department admins.
- **Backend (Node.js / Express):** REST APIs that handle authentication, matching, scheduling, and meeting records.
- **Database (MongoDB):** Stores user profiles, mentor–student assignments, scheduled meetings, and meeting notes.

Since this is a web application, there are no special hardware requirements — it needs to run in a standard browser on both desktop and mobile, which keeps it accessible to students and faculty without asking them to install anything extra.

### 3.2 Module Development

The application is broken into four core modules:

1. **Allocation:** Students are matched to a mentor in their department with a matching academic focus area, assigned on a first-available basis. Admins can see mentee counts per faculty member on their dashboard and manually reassign a student if the load looks uneven — this keeps the matching logic simple and easy to reason about, rather than an automated balancing algorithm.
2. **Scheduling:** Faculty publish their office-hour availability, and students book a slot directly. The server checks for conflicts before confirming any booking, and every booked meeting automatically creates a log entry.
3. **Meeting Notes:** After each meeting, faculty add a short free-text note, building a visible, timestamped history of contact for that student.
4. **Dashboards:** Students see their mentor, meeting history, and notes; faculty see their mentee list and upcoming meetings; admins see a read-only summary of mentorship activity per department — mentee counts, meeting counts, and completion rate.

### 3.3 Execution and Integration

Development happens in short, weekly iterations, with each module — allocation, scheduling, and dashboards — built and demoed on its own before being integrated with the others, so problems surface early instead of at the end. Given the limited timeline, the team is deploying manually to a hosting platform once a module is stable, rather than setting up an automated build/deploy pipeline; this keeps the team's time focused on features that are actually visible in a demo.

### 3.4 Testing and Validation

Once modules are integrated, the system is checked against specific, observable criteria rather than just "does it run":

- **Assignment correctness:** every student is matched only to a mentor within their own department and stated focus area.
- **Meeting completion rate:** the percentage of scheduled meetings that actually get logged as held, versus missed or no-show.
- **Admin visibility:** whether a department admin can answer "how is mentorship going in my department right now" directly from the dashboard, without having to ask faculty manually.
- **User satisfaction:** a short post-pilot survey (1–5 rating) asking students and faculty whether the system made mentorship easier to manage.

Testing is done manually by the team against these criteria, rather than through an automated test suite, given the project timeline. This will be tested with a small pilot group — a handful of students and their assigned faculty mentors within one department — before considering a wider rollout.

### 3.5 Constraints and Safety

The interface has to stay simple enough that faculty — who will not tolerate a clunky tool — actually use it instead of going back to email. There are no physical safety concerns since this is a web application, but there are data-safety concerns worth planning for early: meeting notes are personal academic data, so they need to be restricted to the assigned mentor and department admin only, not visible university-wide. To fit the project timeline, a few features are intentionally left out of this version and treated as possible extensions rather than core scope: automated workload-balancing in the allocation module, email/SMS notifications (the system uses in-app status only), and password-recovery flows (basic login/signup only). None of these affect the core mentorship workflow — they can be added later without changing the system's architecture.

## 4 Timeline

| Week | Task | Deliverable |
|---|---|---|
| 1–2 | Requirements finalization, schema design, project setup | Architecture doc, repo setup |
| 3–5 | Build authentication and the allocation module | Working student–mentor matching |
| 6–8 | Build scheduling module and meeting notes | Bookable office hours, auto-generated logs |
| 9–10 | Build role-based dashboards (student/faculty/admin) | Working dashboards for all three roles |
| 11 | Pilot testing within one department | Pilot feedback, evaluation metrics |
| 12 | **Buffer week:** fixes from pilot feedback, final polish and docs | Final submission |

## 5 Resources and Budget

**Material resources:** No specialized lab equipment is needed. The project uses MongoDB Atlas's free tier for the database and a free-tier hosting platform (e.g., Render or Vercel) for the frontend and backend, with a shared GitHub repository for version control.

**Budget:** No external funding is required. Every tool and service used — database hosting, web hosting, and version control — is available on a free tier, or already provided through the university.